



ITI

Data Management Track

Graduation Project Document



BY:

**Youssef Etman
Ibrahim El-Sadek**

General notes:

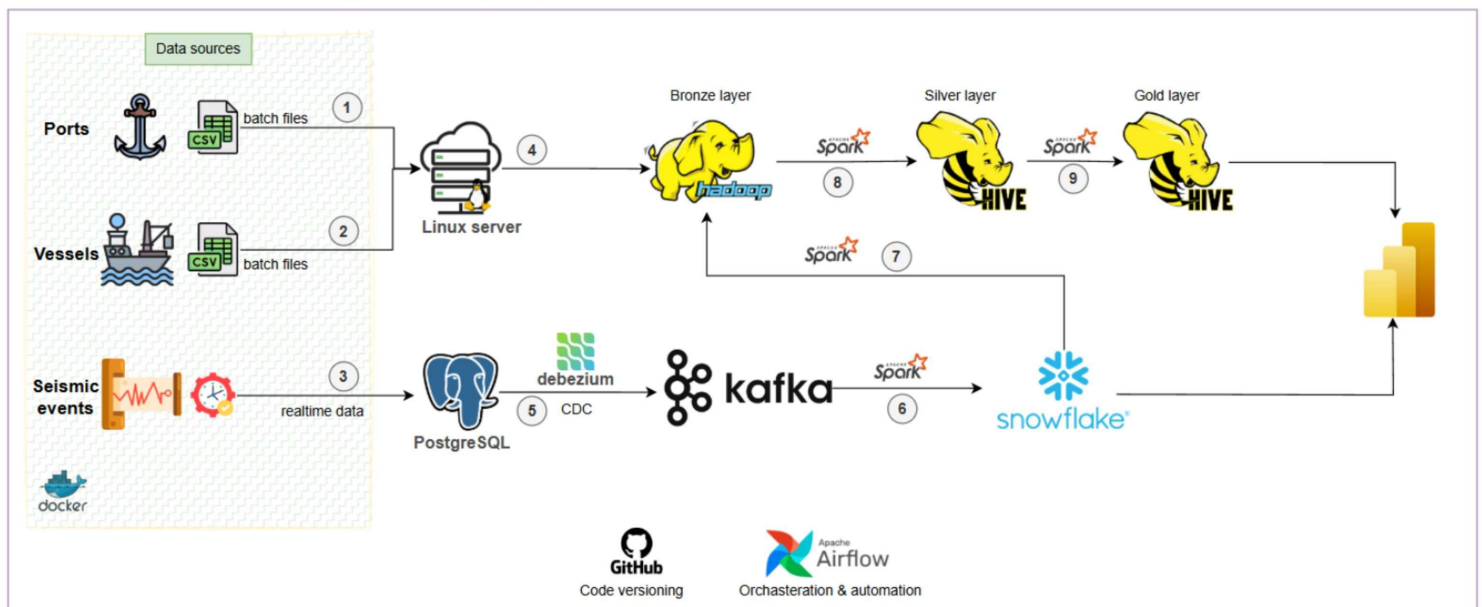
- Technologies/Stack mentioned in the project is the recommended stack, feel free to use different technologies but after approval from the instructor.

Business overview:

The data platform of a company “TidaLine” (Tidal + Line) operating in logistics and energy infrastructure, focusing on maritime activities and seismic risk monitoring. The business requires continuous tracking of port and vessel activities, as well as real-time seismic events, to make strategic and safety-related decisions.

The system must integrate data from multiple sources—some updated periodically (like ports and vessels databases) and others in real-time (such as seismic events). The company wants to build a data lake + data warehouse solution to power business reports and dashboards for different stakeholders.

Technical description of project:



1- Data sources and generation layer:

Source	Type	Format	Frequency	Description	link
Ports Lookup API	CSV over API	CSV	Monthly	Provides metadata for ports, including codes, countries, and geolocation	link
Vessels Website	Web scraping	CSV	Daily	Contains information on vessels, their movements, and estimated arrival	link (we only want cargo and oil tanker vessels)
Seismic Events API	REST	JSON	Real-time	Streams seismic activity from a global monitoring service	link

For each source we need to develop a job that runs based on frequency mentioned in the above table and write data in the next stage. And we need to make this layer **containerized using docker** to help in running it on different environments.

- Step 1: Recommended Python (Pandas).
 - Read the CSV using the exposed API and write to an SFTP Server (Linux server).
- Step 2: Recommended Python (Pandas + requests + BeautifulSoup).
 - Scrape the website provided for Egyptian Oil tankers and Cargo vessels only.
 - You MUST abide by the scraping rules of the website found in the robots.txt file before scraping for two reasons:
 1. We are ethical developers :)
 2. To avoid getting your public IP blocked by the website.
 - Write the scraped data as a CSV file to the same SFTP server.
 - For every vessel you should scrape:
 name,type,year_built,gross_tonnage,deadweight,length(m),beam(m),detail_link,departure_date,last_port_country,last_port_name,arrival_date,destination_port_country,destination_port_name,destination_port_lat,destination_port_lon,reported_status,report_date.
- Step 3: Recommended Python (Check API documentation).
 - Use the public API provided to ingest data real-time into Postgres database.
 - Fields needed:
 source_id,source_catalog,lastupdate,time,flynn_region,lat,lon,depth,evtype,auth,mag,magtype,unid,action.
 - Handle the data differently depending on the “action” of the event.
- Steps 1 & 2 should be scheduled based on the frequency mentioned above. You can use crontab or Airflow, but you need to make sure that your scripts are monitored & logged because you must be notified in case of failure. While Step 3 should be a long running job (running all the time) and also must have a means of logging and monitoring.

2- Extraction layer:

- Step 4: Recommended Scripting language (Bash or Python).
 The SFTP servers have small storage compared to HDFS, that is why They are used as an intermediate step between different systems (a jump node). Extract data from **Linux Server** to **HDFS (Data Lake)**, the job should be scheduled using **Airflow**.
- Step 5: Recommended Debezium + Kafka.
 We need to avail the seismic events data in real-time to the BI team so they can build real-time dashboards and support in risk analysis. It's prohibited for the BI team to build dashboards on top of the operational OLTP system. So in this step we will build an Operational Data Store (ODS) that gives access for the BI team to real-time operational data without over-loading our OLTP system. Changes to the Postgres seismic events operational table should be reflected in real-time to our Snowflake ODS using **CDC** (Change Data Capture).
 - Use Debezium to connect to Postgres and listen to changes on the seismic events table.
 - Log changes to Kafka topic.

3- Processing & storage:

- Step 6: **Must use** Spark Streaming.
 - Spark streaming job that extract the data from Kafka and write it to ODS (Snowflake).
 - ODS will be used by BI team to build dashboards in PowerBi.
- Step 7: **Must use** Spark.
Daily data written to snowflake should be archived to HDFS using spark job.
- Step 8: **Must use** Spark.
Raw data from Bronze layer should be cleaned and moved to Silver (Staging) layer.
- Step 9: **Must use** Spark.
 - Apply business logic to data stored in Silver layer and move it to Gold (Business) layer.
 - This layer will be used by BI team to build dashboards in PowerBi.

4- Reporting:

In this part we need to ensure that we have a good modeling that serve business requirements effectively and in optimized way from storage to query runtime and complexity of queries, also consider that we may have a new business requirement later after building your model.

5- Code versioning:

- a) Create a git repo for the project with an appropriate descriptive name.
- i. Start documenting your version control workflow.

An example of a workflow that you can use.

Example 4-1. Template workflow

```
Product Manager: Name
Dev site: URL
Branch deployed on dev site: name of branch
Live site: URL
Branch deployed on live site: name of branch
When starting a dev ticket, branch from: name of branch
When starting a hotfix ticket, branch from: name of branch
When updating your work, use: git command
When merging your work post review, use: git command
```

- ii. Choose a branching strategy that suits your use case (e.g. GitHub Flow, GitLab Flow, Git Flow).
- iii. Branches Constraints:
- i. Each feature branch should have a code for only one ticket or feature.
 - ii. integration, dev, staging, or preprod branches (if any of them exists) should be used only for bringing pieces of code together.
 - iii. main/master branch should have a working and stable version of your project.
- iv. Put a review strategy for the code before merging it into any of the integration branches to ensure that no merge conflicts exist, or the documentation is missing.

- b) Use GitHub as a hosting platform and all of you should be collaborators in it.
 - i. Each ticket should be represented as an issue on GitHub, and the commits related to this ticket should be attached to the issue as well.
 - ii. Document each pull request, issue, and the whole project in the README file.
- c) Use GitHub Actions for automating the testing and deployment processes of your code.
- d) Ensure using best practices and normalize naming conventions among your team members.

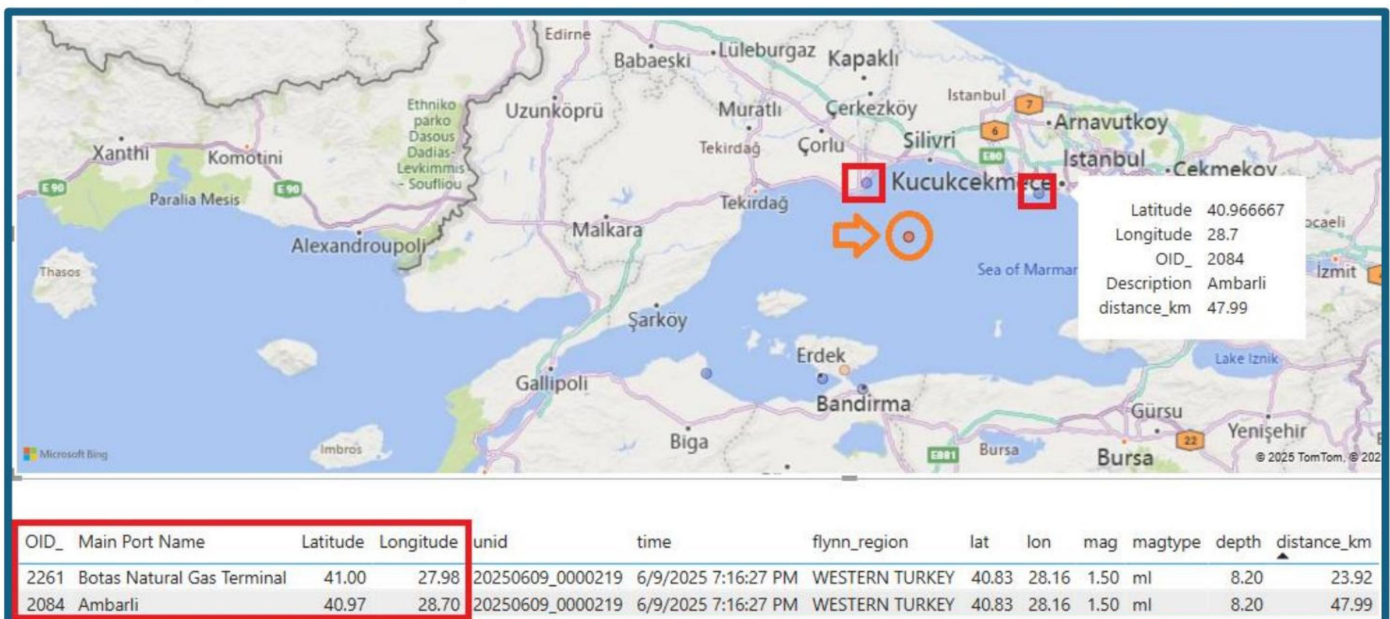
Technical requirements:

- All scripts should be logged & monitored.
- All scripts should be clean & well commented.
- Your scripts should be well tested before getting deployed.
- There should be a means of notifying the support team in case of failure in any step.
- Consider recovery scenarios for job failure, and how you will recover failed jobs.
- Your pipelines must be idempotent to ensure safe backfills.
- Ensure reproducibility, automation, and version control using **Airflow** and **Git**.
- Add metadata columns if needed, ex: insertion_date.
- **In each layer, ensure that data is partitioned in appropriate way!**
- Before building your data model you should study the business very well to have a complete understanding of what you need to implement.
- You should plan and track your project using a Project Management tool. Recommended Trello board with 4 lists (ToDo, In Progress, Blocking & Done). All 4 members must log in to the board using their personal emails. Each member must enter his/her task by him/her self so it is clear what is everyone's task.

Business requirements:

a) Risk Analysis:

- It is required to be able to see the earthquakes' locations on the map in **real-time (last 24 hours)** as well as the ports that are nearby.



- You must differentiate between the ports (here in blue) and the seismic events (here in orange) on the map.

- When clicking on an earthquake data point on map the below table should show the nearby ports and vice-versa when clicking on port the table should show the nearby earthquakes.
- The user should be able to determine the maximum distance below which the earthquake is considered to be close.

- It is also required to have a similar map but for historical events. For example, look at the seismic events that happened 2 days ago and the ports that are at least 50km away from an event.
- Classify the ports into 3 categories (safe, cautionary, dangerous). You need to think of a KPI upon which you will apply your classification.
- The Health & Safety team needs to be able to filter on dangerous ports and see the vessels that are headed their way to be able to notify them to change route.

b) Vessels Analysis:

- Business team would like to have an overview on our company's vessels from the point of view of their type, weight, size, delay in reporting their status, etc ...

c) Trips Analysis:

- Analyze times of departure and arrival, delay in arrival, destination countries, etc ...

d) Ports Analysis:

- We aim to enrich port data by introducing a new classification flag named `supplies_rate`, which evaluates the availability of critical supplies at each port, The following key supplies are considered: Provisions, Fuel Oil, Diesel, Potable Water, Repairs, based on the combination of available supplies, ports will be classified as follows:

Available Supplies Count	Required Supplies	supplies_rate Classification
All 5 available	Provisions, Fuel Oil, Diesel, Potable Water, Repairs	Excellent
Any 3-4 available	At least 3 of the above	Good
Any 1-2 available	At least 1 of the above	Limited
None available	-	Unavailable

- Like The previous logic we need to apply it on communication channels available and classify it: Radio, telephone, airport, Telefax

Available Communication Count	Required channels	comm_rate Classification
All 4 available	all	Excellent
Any 2-3 available	At least 2 of the above	Good
Any 1 available	At least 1 of the above	Limited
None available	-	Unavailable

- Business team would like to have an overview on the ports worldwide from the point of view of country, depth, harbor size & type, shelter, `supplies_rate`, `communication_channels`, etc ...

Team Structure and Responsibilities

Team Size: Each team will consist of 4 members.

1. One Team Leader:

- Coordinating tasks and ensuring on-time delivery.
- Being the main point of contact with the mentoring/instruction team.
- Overseeing version control and pull request approvals.

2. Two Business Analysts:

- Responsible for building and understanding the business model of the company.
- Build an understanding of maritime logistics, ports and vessels.
- Study seismic activity and earthquakes.
- Use the internet, read scientific publications, contact domain experts, etc...
- Responsible for dashboard ideas and answering business questions.
- Can search and add data from sources other than those mentioned here to enrich your project.

3. One Solution Architect:

- Responsible for studying the project architecture.
- Study all the tools recommended by the instructor.
- Research other alternative tools/solutions and compare them.
- Responsible for suggesting and supervising new POCs (Proof of concept).

Ofcourse all the 4 members should participate in code development, infrastructure, automation, ... Also, feel free to work with your teammate in his/her designated task so we can all share the experience.

Tips & Recommendations:

- Try to finish the vessels scraping first in order to have a large dataset of vessels historical data when building the dashboard at the end.
- Try to finish ingesting the seismic events changes to ODS for the same reason as above.
- A docker-compose file for (Postgres + Debezium + ZooKeeper + Kafka) will be provided.
- Feel free to enrich your project with other data sources than those specified here.
- Feel free to research other tools/solutions than those recommended here.

Bonus:

- Create a POC for replacing any of the recommended tools, ex:
 - Replace PowerBi with another tool (Tableau, Superset, ...)
 - Replace Debezium.
 - Replace Airflow with another orchestration tool.
 - ...

Instructors Follow up methodology:

- First meeting will be for explaining the project data, solution, technical requirements, business requirements, team structure, grading, etc ...
- Follow up meetings with each team will be every (), these meetings are only for support and there is no grading included.
- Each follow up meeting will have:
 - Receive updates and current status for each team.
 - Provide support if needed.
 - Request additional business requirements.

Project Milestones:

Phase	Milestone
1	Data sources extraction.
2	Data Understanding (Understand meaning of all fields & KPIs) + Business research.
3	CDC + ODS
4	Batch ingestion into data lake different layers.
5	Initial Dashboard.
6	Final delivery.

- Please note that these phases are for milestone delivery only and don't represent the actual steps. For example, you may need to work on phases 3 & 4 together in parallel to finalize the ODS part.
- In each milestone we will have a look at: docker, airflow, github, Trello.

Delivery Methodology

Each team must deliver the following as part of the final submission:

1. End-to-End Running Flow

- A complete, demonstrable flow starting from **data ingestion**, followed by **processing/transformation**, ending with **dashboard/report generation**.
- All components should be integrated and runnable as a pipeline.

2. Presentation File

- A structured **PowerPoint or PDF** presentation.

3. Live Demo

- A **real-time demo** of the running pipeline during the final presentation session.
- The demo should cover critical stages and explain how data flows across components.

4. GitHub Repository

- A private **GitHub repository** containing:
 - All **source code**
 - Configuration and environment setup files
 - Any **data generator** or test data
 - The final presentation
 - Instructions to run the project locally or in a containerized setup (e.g., via docker-compose, notebooks, or bash scripts)

5. Trello board link

- Four lists (ToDo, In Progress, Blocking, Done).
- **Each member should add his own cards to each list.**
- In planning phase, start adding tasks to the ToDo list.
- In execution phase, start moving tasks to the In Progress list.
- Once the task is completed, move it to the Done list and mark it as complete.
- Tasks that aren't completed due to an error/challenge are moved to the blocking list.
- You can move tasks both ways ToDo → In Progress → Done or vice versa.
- The final delivery can have tasks in the In Progress list or the ToDo list for tasks that weren't completed or future tasks. However, most of the tasks should be in the Done list.

6. Project Documentation

- Detailed documentation for every step in the project.

Grading:

Group grade:

- The status of the final submission. (Running end-to-end, handles failures in pipeline, pipelines are idempotent, good design for code, tables, dashboards, github history, Trello board, etc...)
- Teamwork during the project.
- Final presentation as a team.

Individual grade:

- Your efforts in your designated role.
- Your efforts in code development, automation, contribution to the github repo, ...

Teams:

Team 1	Ahmed Waheed, Mai Serry, Shahd , Ehab Nasr
Team 2	Salah, Ayman , Roaa
Team 3	Omar Mostafa, Mohamed Shaker , Farah, Salma-Algayar
Team 4	Asem, Afandi, Farida, Ahmed-Yehia
Team 5	Sara-Tamer, Omar-Ayman , Zeyad, Amira
Team 6	Elsayed, Mohamed Tarek, Ahmed-Metwly, Mariam-Taha